

1.1.1 Aims

The overarching aim of this project is to design, implement, and evaluate an agentic [LLM](#) assistant that operates directly on Kaggle computational notebooks within Google Chrome. Specifically, the work aims to:

- provide conversational assistance grounded in scraped notebook state rather than user-copied code alone;
- expose a bounded tool interface through which the model can inspect and modify cells in the live editor;
- enforce predictable agentic behaviour through a mandatory two-phase query–implement protocol and host-side batch reordering; and
- demonstrate feasibility on representative Kaggle /edit sessions relevant to data science coursework and competition workflows.

1.1.2 Research Objectives

The following research objectives guide the design and evaluation of the system:

1. **Architecture.** Specify and implement a Chrome extension and Python native messaging host that communicate securely, route chat by mode (Ask, Code, Agentic), and associate each request with the active notebook URL and browser tab identifier.
2. **Context pipeline.** Develop a DOM-scraping pipeline that serialises notebook cells into live and persistent JSON snapshots, enabling the host to inject current structure and content into [LLM](#) prompts without manual copy-and-paste.
3. **Tool-mediated cell operations.** Implement and register browser tools with session URL and tab_id coercion so dispatches target the correct editor instance. Mutation tools are `insert_cell`, `edit_cell_by_index`, `delete_by_index`, and `run_cell`. Read tools are `notebook_get_cell` and `list_cells`.
4. **Agentic execution contract.** Integrate GLM 4.7 native `tool_calls` (Cerebras) under a mandatory two-phase protocol (read-only query, then implement), a maximum of two [API](#) calls per user message, fire-and-forget batch dispatch, and host-side descending reorder for bulk delete and insert operations.